완료 보고서

SPI & I2C RTL 설계 및 **UVM** 기반 검증

한승훈

2026.06.11 ~ 2026.06.18

목차

1. 개요 (Overview)
  1.1 목적 및 목표
  1.2 프로젝트 요약
  1.3 검증 대상 및 설계 사양 요약
2. 프로젝트 관리 (Project Management)
  2.1 일정 계획
  2.2 역할 분담
  2.3 개발 및 검증 환경
3. 아키텍처 설계 (Architecture)
  3.1 Verification System Architecture
  3.2 UVM Verification Architecture
  3.3 SPI/I2C 프로토콜 배경
4. 상세 설계 (Detailed Design)
  4.1 RTL Design Under Test
  4.2 SPI UVM Verification Environment
  4.3 I2C UVM Verification Environment
  4.4 FPGA Validation Setup
5. 시뮬레이션 및 검증 (Simulation & Verification)
  5.1 검증 계획 및 시나리오
  5.2 SPI 검증 결과
  5.3 I2C 검증 결과
  5.4 Waveform 분석
6. 결과 분석 및 트러블슈팅
  6.1 SPI Coverage 분석
  6.2 I2C ACK Sampling Issue
  6.3 개선 결과
7. 결론 및 고찰

개요 (Overview)

1.1 목적 및 목표

본 프로젝트는 SPI 및 I2C Master/Slave RTL을 대상으로 UVM 기반 검증 환경을 구축하고, 프로토콜 특성에 맞는 transaction-level 검증을 수행하는 것을 목적으로 한다. 단순한 파형 확인을 넘어 Sequence, Driver, Monitor, Scoreboard, Functional Coverage로 구성된 검증 구조를 설계하고, 검증 결과와 coverage hole을 분석하는 데 중점을 두었다.

- SPI Mode 0 및 I2C 100 kHz 동작을 대상으로 Master/Slave RTL의 정상 transaction 수행 여부를 검증한다.
- Monitor가 실제 bus signal을 transaction으로 복원하고, Scoreboard가 expected / actual data를 비교하도록 UVM 환경을 구성한다.
- Functional Coverage를 이용하여 data pattern, address, operation, ACK/NACK 등 주요 검증 항목의 도달 여부를 확인한다.
- Simulation 결과를 Basys3 board-to-board 환경 및 Logic Analyzer 파형과 비교하여 실보드 검증까지 수행한다.

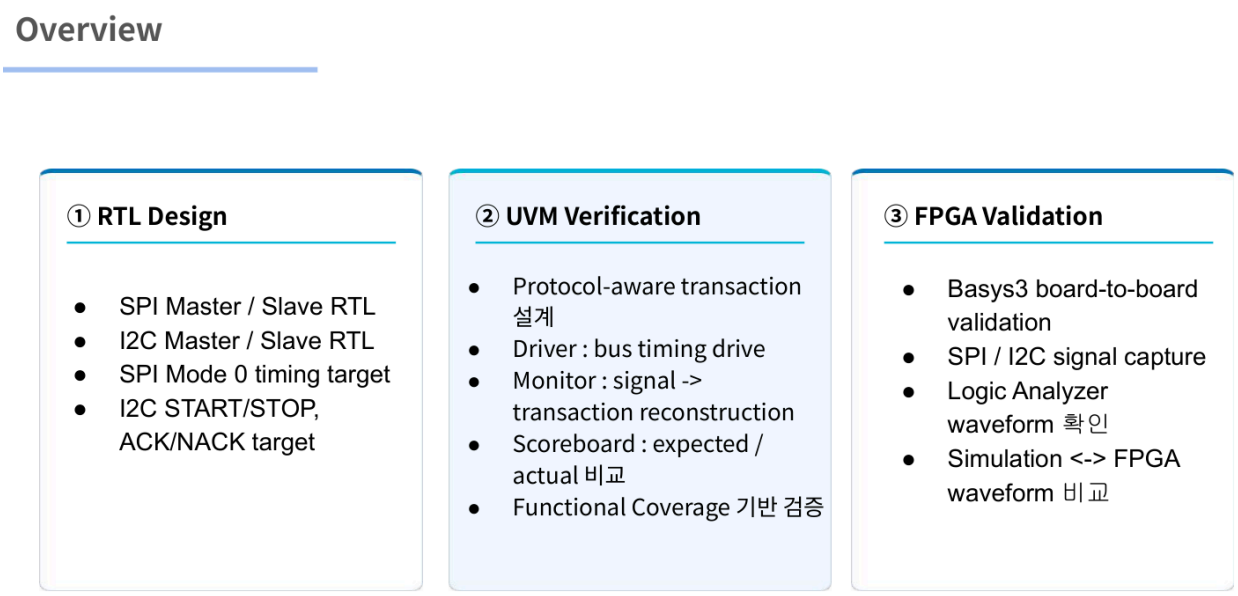


그림 1. 프로젝트 Overview - RTL Design, UVM Verification, FPGA Validation

1.2 프로젝트 요약

프로젝트는 SPI/I2C RTL 설계, UVM 검증 환경 구축, FPGA board-to-board validation의 세 단계로 진행되었다. RTL 설계는 SPI Master/Slave 및 I2C Master/Slave를 대상으로 하며, 검증의 중심은 UVM 기반 자동 검증 환경이다. SPI는 Full-Duplex 데이터 전송을 중심으로 검증하였고, I2C는 START/STOP, Address, ACK/NACK, WRITE/READ data path를 중심으로 검증하였다.

구분	주요 내용
RTL Design	SPI Master/Slave RTL, I2C Master/Slave RTL 구현 및 protocol timing target 정의
UVM Verification	Protocol-aware transaction, Driver bus timing drive, Monitor transaction reconstruction, Scoreboard 비교, Functional Coverage 수집
FPGA Validation	Basys3 이중 보드 구성, SPI/I2C signal capture, Logic Analyzer 파형 확인, Simulation 결과와 비교

1.3 검증 대상 및 설계 사양 요약

검증 대상은 SPI Master/Slave RTL과 I2C Master/Slave RTL이다. SPI는 4-wire interface와 Full-Duplex 전송 구조를 가지며, I2C는 Open-Drain 기반의 2-wire Half-Duplex bus 구조를 가진다. 두 프로토콜은 transaction 경계와 bus drive 방식이 다르므로, Monitor와 Scoreboard의 관찰 기준도 각각 분리하여 설계하였다.

항목	SPI	I2C
Interface	SCLK, MOSI, MISO, CS_N	SCL, SDA
Bus / Duplex	4-wire, Full-Duplex	Open-Drain with Pull-up, Half-Duplex
동작 조건	Mode 0, CPOL=0, CPHA=0	Standard Mode 100 kHz
Data Width / Address	8-bit data	7-bit address, target 7'h12
Transaction 경계	CS_N Low -> 8-bit transfer -> CS_N High	START -> ADDR -> ACK -> DATA -> ACK/NACK -> STOP
검증 핵심	Master TX == Slave RX, Slave TX == Master RX	WRITE/READ data match, address match, 9th SCL ACK/NACK

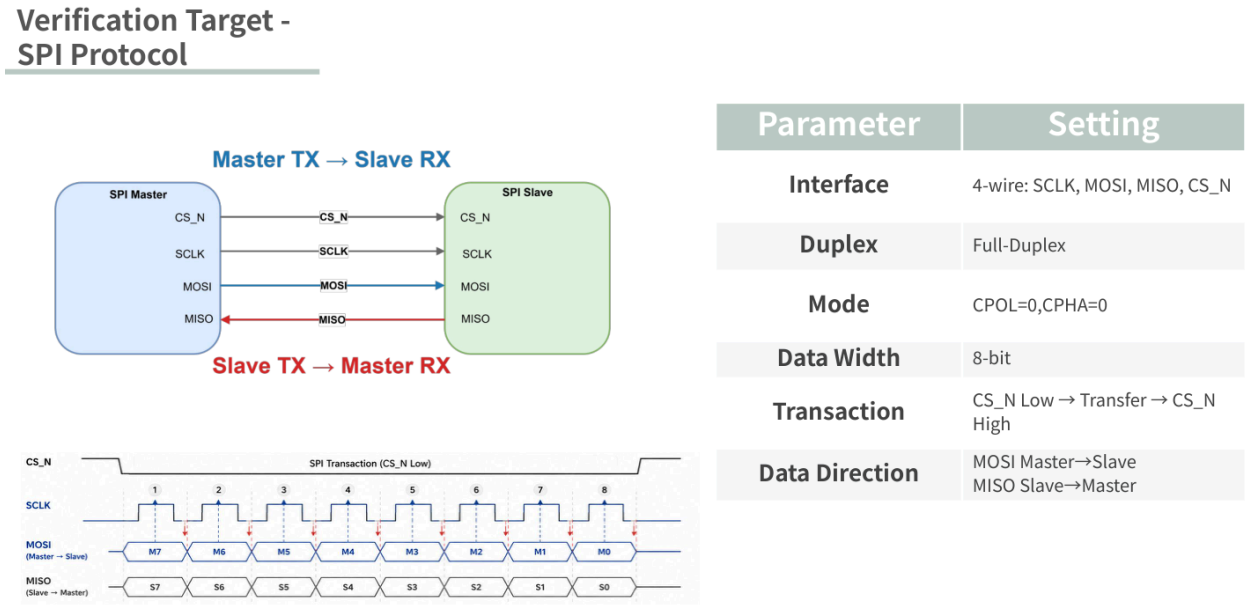
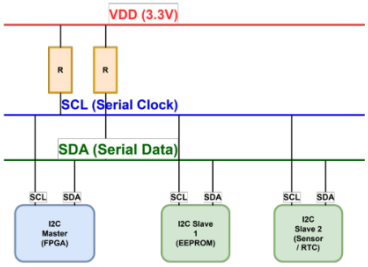


그림 2. SPI Protocol 검증 대상 및 사양

Verification Target - I2C Protocol



Parameter	Setting
Interface	2-wire: SCL, SDA
Bus Type	Open-Drain with Pull-up
Duplex	Half-Duplex
Speed	100 kHz
Target Address	7'h12
ACK/NACK	9th clock, ACK=0 / NACK=1

I2C 1-Byte Write Operation (쓰기 동작)



I2C 1-Byte Read Operation (읽기 동작)



그림 3. I2C Protocol 검증 대상 및 사양

2. 프로젝트 관리 (Project Management)

2.1 일정 계획 (Schedule)

단계	기간	주요 작업
요구사항 분석	Day 1	SPI/I2C protocol specification 확인, 검증 목표 및 coverage target 정의
RTL 설계	Day 1~2	SPI Master/Slave, I2C Master/Slave RTL 구현 및 interface 정리
UVM Architecture 설계	Day 2	Sequence, Driver, Monitor, Scoreboard, Coverage 구조 설계
SPI 검증	Day 3	SPI full-duplex transaction, MOSI/MISO data match, coverage 수집
I2C 검증	Day 3~4	I2C WRITE/READ, address match, ACK/NACK, wrong address scenario 검증
Troubleshooting	Day 4~5	I2C ACK sampling issue 원인 분석 및 Monitor sampling 개선
FPGA Validation	Day 5~6	Basys3 board-to-board 연결, Logic Analyzer waveform 확인
Documentation	Day 6~7	발표자료 및 완료보고서 작성, 결과 정리

2.2 역할 분담 (Roles & Responsibilities)

역할	담당 업무
RTL 설계	SPI/I2C Master/Slave RTL 구현, interface 연결, FPGA validation용 top 구성
Verification	UVM testbench 구조 설계, sequence/driver/monitor/scoreboard/coverage 구현, VCS simulation 및 결과 분석
Debug	ACK sampling issue, coverage hole, protocol timing mismatch 분석 및 수정
FPGA 검증	Basys3 이중 보드 연결, Logic Analyzer를 통한 실제 pin waveform 확인
문서화	검증 계획, 결과표, waveform 분석, troubleshooting 및 conclusion 정리

2.3 개발 및 검증 환경

분류	내용
설계 언어	Verilog HDL, SystemVerilog
검증 방법론	UVM
Simulator	VCS / Verdi 기반 simulation 및 waveform debug
FPGA Tool	Xilinx Vivado
FPGA Board	Basys3 board 2대, Master/Slave board-to-board 구성
측정 장비	Logic Analyzer
주요 검증 산출물	UVM log, Scoreboard pass/fail, Functional Coverage, waveform screenshot, FPGA 연결 사진

3. 아키텍처 설계 (Architecture)

3.1 Verification System Architecture

전체 검증 구조는 SPI와 I2C를 각각 독립적인 protocol-aware agent로 구성하고, 공통적인 UVM 환경 안에서 test sequence를 실행하는 형태로 설계하였다. 각 agent는 Driver를 통해 DUT에 stimulus를 인가하고, Monitor가 bus signal을 관찰하여 transaction으로 복원한다. 복원된 transaction은 Scoreboard와 Functional Coverage로 전달되어 데이터 일치 여부와 scenario 도달 여부를 확인한다.

UVM Architecture

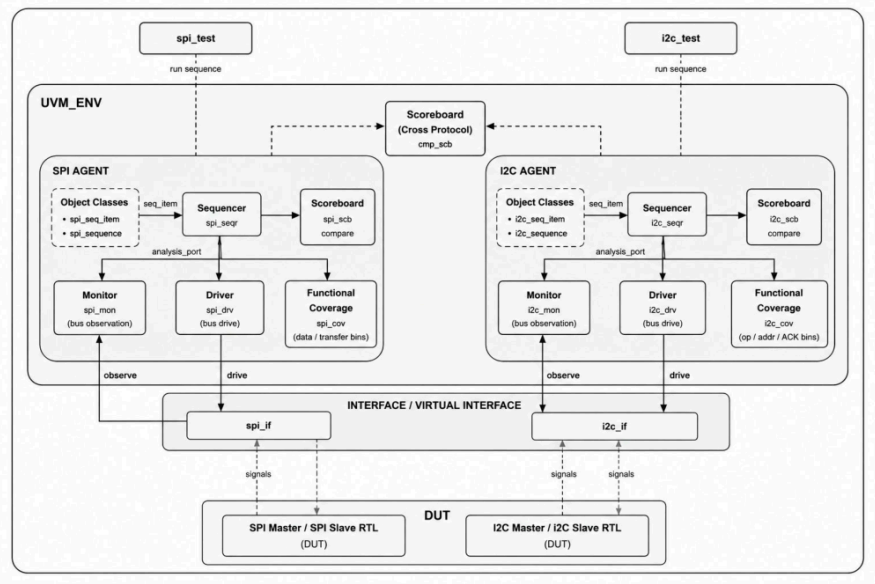


그림 4. UVM Verification Architecture

3.2 UVM Verification Architecture

UVM Component	역할	본 프로젝트 적용 내용
Sequence / Sequencer	검증 scenario 및 transaction 생성	SPI master_tx/slave_tx pattern, I2C WRITE/READ command 및 address/data 생성
Driver	Transaction을 pin-level stimulus로 변환	SPI start 및 bus timing drive, I2C cmd_start/cmd_write/cmd_read/cmd_stop drive
DUT	검증 대상 RTL	SPI Master/Slave RTL, I2C Master/Slave RTL
Monitor	Bus signal을 관찰하여 transaction으로 복원	SPI CS_N/SCLK/MOSI/MISO sampling, I2C START/STOP/ADDR/ACK/DATA decode
Scoreboard	Expected data와 actual data 비교	SPI full-duplex data match, I2C WRITE/READ data match 및 address match 확인
Functional Coverage	검증 scenario 도달 여부 확인	Data pattern, operation, address, ACK/NACK, cross coverage 수집

3.3 SPI/I2C 프로토콜 배경

SPI와 I2C는 모두 저속 주변장치 제어에 널리 사용되는 직렬 통신 프로토콜이지만, 검증 관점에서는 transaction 경계와 data sampling 기준이 다르다. SPI는 CS_N이 Low인 구간을 transaction window로 보고 SCLK 기준 8-bit 전송이 이루어지는지 확인한다. 반면 I2C는 START/STOP condition, 7-bit address, 9번째 SCL에서의 ACK/NACK 판별, SDA drive/release 여부가 핵심 검증 포인트가 된다.

Verification Focus

Verification Point	SPI (Full-Duplex)	I2C (Open-Drain)
Transaction 경계	CS_N Falling → Transfer → CS_N Rising	START / STOP condition
Data Integrity	Master TX == Slave RX Slave TX == Master RX	WRITE: Slave RX == Master TX READ: Master RX == Slave TX
Handshake	No ACK, CS_N based	ACK (SDA Low) / NACK (Release)
Enable / Target	Transfer only when CS_N=0	7-bit Address Match (7'h12)
Bus Drive	물리적 분리 (MOSI/MISO)	SDA Drive (Low / Release)

그림 5. SPI/I2C Verification Focus

4. 상세 설계 (Detailed Design)

4.1 RTL Design Under Test

DUT는 SPI Master/Slave RTL과 I2C Master/Slave RTL로 구성된다. SPI DUT는 Mode 0 timing을 기준으로 CS_N Low 구간에서 8 SCLK 동안 MOSI와 MISO가 동시에 이동하는 Full-Duplex 구조이다. I2C DUT는 Open-Drain bus 특성을 반영하여 SDA를 직접 1로 drive하지 않고, release와 pull-up을 통해 High 상태를 형성하는 구조를 갖는다.

DUT	주요 기능	검증 포인트
SPI Master	CS_N 제어, SCLK 생성, MOSI 전송, MISO 수신	8 SCLK cycle, MOSI/MISO data path, busy/done timing
SPI Slave	CS_N active 구간에서 MOSI 수신 및 MISO 응답	Master TX 수신값, Slave TX 응답값, 첫 bit drive timing
I2C Master	START/STOP 생성, address/write/read command 수행	START/STOP condition, ACK/NACK sampling, read/write data path
I2C Slave	Target address match, ACK 응답, write data 수신, read data 제공	7'h12 address match, SDA drive/release, ACK/NACK response

4.2 SPI UVM Verification Environment

SPI UVM 검증은 Sequence에서 master_tx와 slave_tx 값을 생성하고, Driver가 start pulse 및 SPI bus timing을 인가하는 방식으로 구성하였다. Monitor는 CS_N Low 구간을 감지한 뒤 SCLK 8 cycle 동안 MOSI와 MISO를 sampling하여 transaction으로 복원한다. Scoreboard는 master_tx가 slave_rx로, slave_tx가 master_rx로 정상 전달되었는지 비교한다.



그림 6. SPI UVM Verification Flow

구분	내용
Observed Signals	CS_N, SCLK, MOSI, MISO, master_rx, slave_rx, busy, done
Expected Model	expected_slave_rx = master_tx_data / expected_master_rx = slave_tx_data
Coverage Target	master_tx/slave_tx data bins, CS_N active window, 8 SCLK cycle transfer, full-duplex data path
Simulation Result	Scoreboard PASS 48 / FAIL 0, UVM_ERROR 0 / UVM_FATAL 0, Coverage 59.59%

4.3 I2C UVM Verification Environment

I2C UVM 검증은 WRITE/READ operation, target address, data pattern, ACK/NACK event를 중심으로 구성하였다. Driver는 cmd_start, cmd_write, cmd_read, cmd_stop 명령을 통해 I2C transaction을 발생시키고, Monitor는 START/STOP condition, ADDR+R/W decode, ACK/NACK event, SDA driver/release 상태를 관찰한다. Scoreboard는 WRITE 시 slave_rx와 write_data를 비교하고, READ 시 master_rx와 slave_tx_data를 비교한다.



그림 7. I2C UVM Verification Flow

구분	내용
Observed Signals	SCL, SDA, START/STOP, ADDR, ACK/NACK, addr_match, rw_mode, SDA driver/release
Expected Model	WRITE: expected_slave_rx = write_data / READ: expected_master_rx = slave_tx_data
Coverage Target	WRITE/READ, target/wrong address, data pattern, ACK/NACK, op x addr x data cross
Simulation Result	Initial: PASS 12 / FAIL 0, Coverage 85.71%, UVM_ERROR 0 / UVM_FATAL 0

4.4 FPGA Validation Setup

Simulation에서 검증한 protocol transaction이 실제 FPGA pin-level waveform에서도 동일하게 관찰되는지 확인하기 위해 Basys3 보드 2대를 Master/Slave로 구성하였다. SPI는 SCLK/MOSI/MISO/CS_N을 연결하고, I2C는 SCL/SDA를 연결하였다. Logic Analyzer를 사용하여 실제 pin waveform을 캡처하고 Simulation 결과와 비교하였다.

FPGA Validation Setup

① Master Board

- 버튼/스위치 입력으로 SPI 또는 I2C transaction 시작
- sw[7:0] 값을 TX data로 사용
- 수신한 RX data를 LED/FND로 표시

② Slave Board

- Master가 전송한 데이터를 수신
- Slave sw[7:0] 값을 response data로 제공
- 수신 결과 및 상태 신호를 LED/FND로 표시

③ Board-to-Board Connection

- SPI: SCLK / MOSI / MISO / CS_N 연결
- I2C: SCL / SDA 연결
- Logic Analyzer로 실제 pin waveform 측정
- Simulation 결과와 FPGA 동작 비교

그림 8. FPGA Validation Setup

Implementation

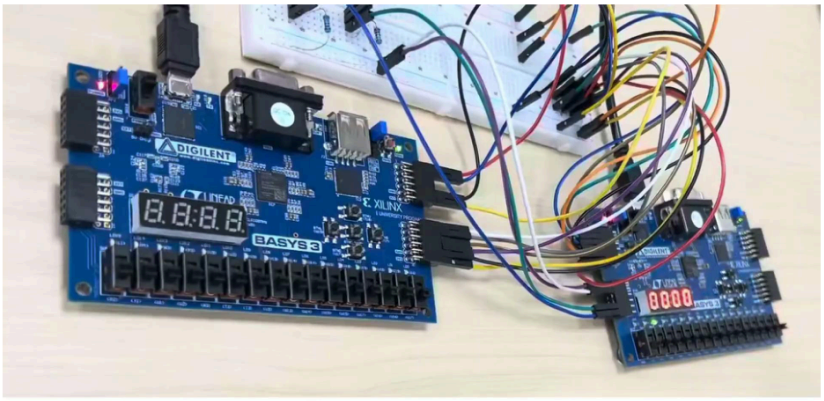


그림 9. Basys3 board-to-board implementation

5. 시뮬레이션 및 검증 (Simulation & Verification)

5.1 검증 계획 및 시나리오

검증은 정상 동작 scenario와 예외 scenario를 구분하여 수행하였다. 정상 scenario는 SPI full-duplex transfer와 I2C WRITE/READ transaction을 중심으로 하였고, 예외 scenario는 I2C wrong address 및 NACK 발생 여부를 중심으로 구성하였다. 검증 결과는 Scoreboard pass/fail, UVM severity report, Functional Coverage, waveform으로 확인하였다.

Protocol	Scenario	Check Point	Expected Result
SPI	Full-duplex transfer	CS_N Low, 8 SCLK cycles, MOSI/MISO sampling	Master TX == Slave RX, Slave TX == Master RX
SPI	Data pattern test	0x00, 0xFF, 0x55, 0xAA 및 pattern/random data	Data bin hit 및 Scoreboard PASS
I2C	Write transaction	START -> ADDR+W -> ACK -> DATA -> ACK -> STOP	slave_rx_data == write_data
I2C	Read transaction	START -> ADDR+R -> ACK -> DATA -> NACK -> STOP	master_rx_data == slave_tx_data
I2C	Negative scenario	Wrong address, NACK event	Address mismatch 및 NACK path 확인
FPGA	Board-to-board validation	Logic Analyzer signal capture	Simulation waveform과 실제 pin waveform 비교

5.2 SPI 검증 결과

SPI 검증 결과 Scoreboard 기준 PASS 48, FAIL 0을 확인하였다. Master TX가 Slave RX로, Slave TX가 Master RX로 전달되는 Full-Duplex data path가 정상적으로 동작하였으며, UVM_ERROR와 UVM_FATAL은 발생하지 않았다. Functional Coverage는 59.59%로 측정되었고, 일부 data pattern bin이 미도달한 coverage hole이 확인되었다.

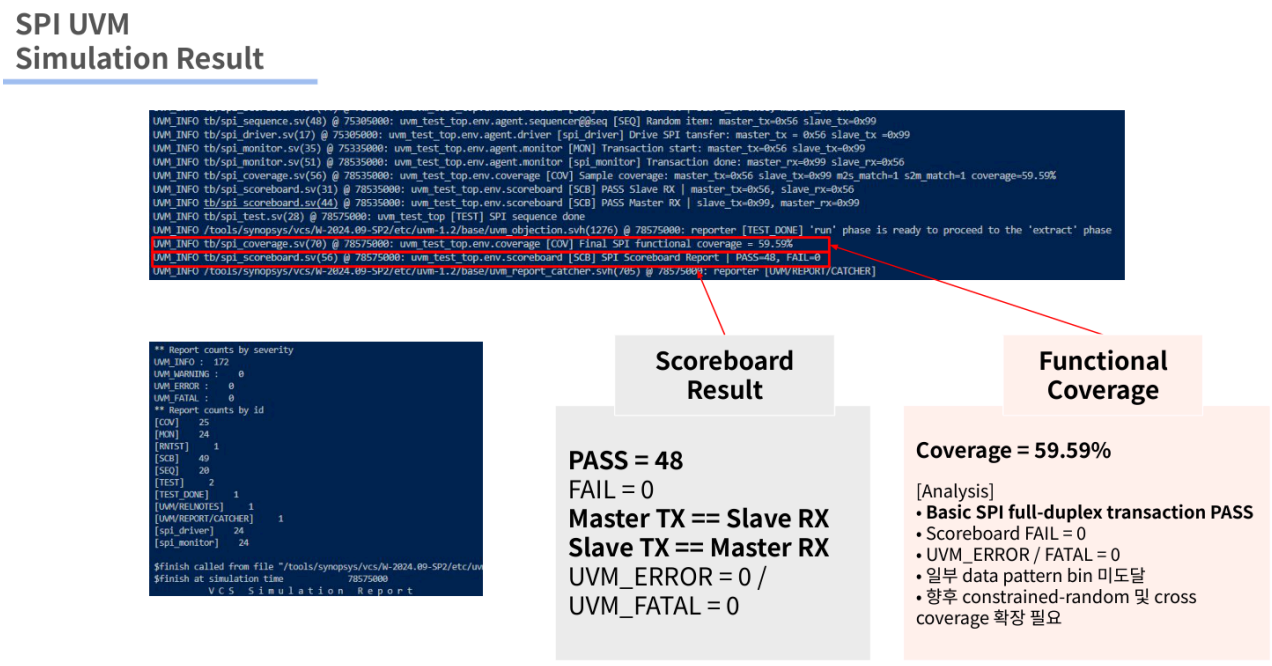


그림 10. SPI UVM Simulation Result

항목	결과	해석
Scoreboard	PASS 48 / FAIL 0	Full-Duplex data match 정상

UVM Severity	UVM_ERROR 0 / UVM_FATAL 0	Fatal-level simulation failure 없음
Functional Coverage	59.59%	Directed pattern 중심으로 일부 data bin 미도달
개선 방향	Constrained-random 및 cross coverage 확장	Coverage hole 분석 기반 추가 검증 필요

5.3 I2C 검증 결과

I2C 초기 검증 결과 WRITE/READ data path는 Scoreboard 기준 PASS 12, FAIL 0으로 정상 확인되었다. Operation, address, data, cross coverage는 대부분 도달하였으나 ACK/NACK coverage가 0%로 남아 있었고, ACK field가 X로 전달되는 UVM_WARNING이 발생하였다. 이는 data path failure가 아니라 Monitor의 ACK sampling 기준 문제로 분류하였다.

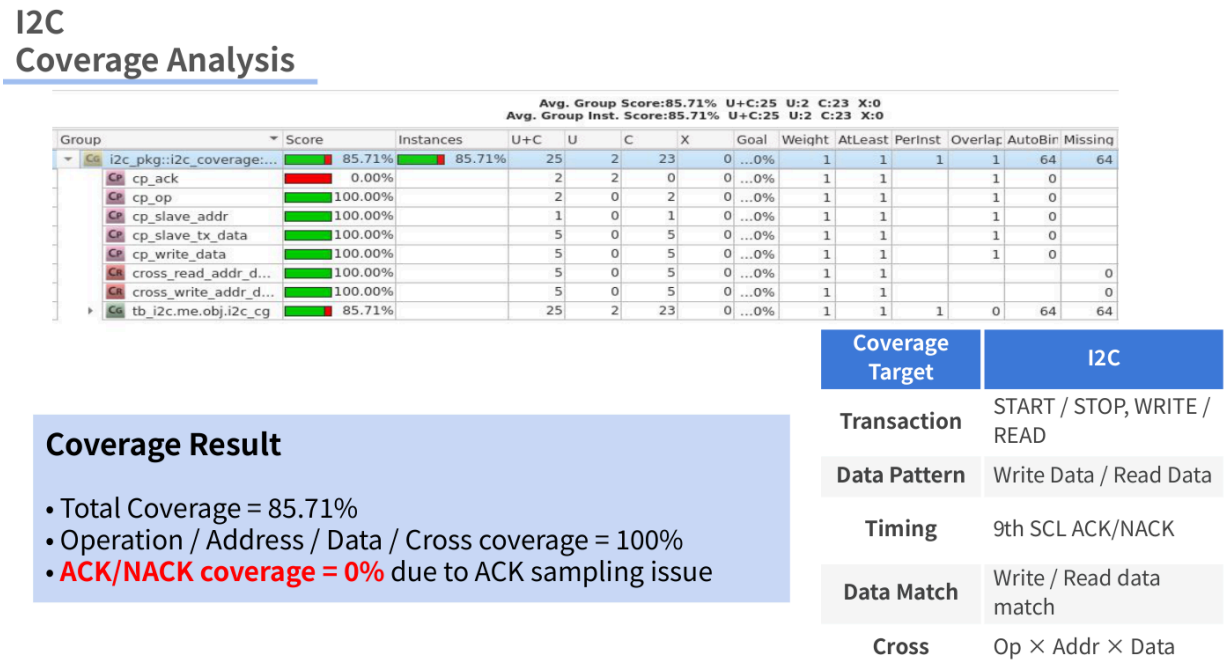
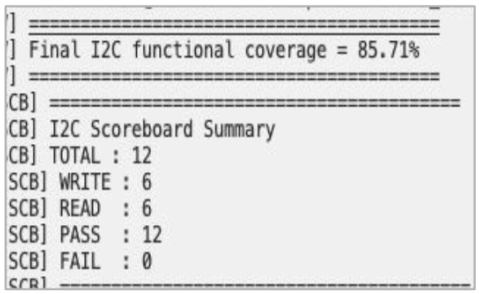
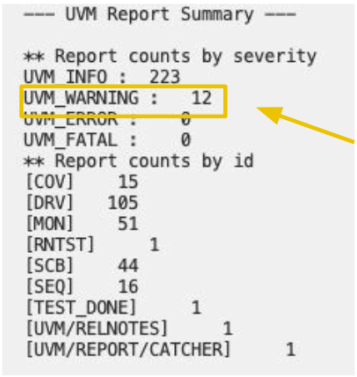


그림 11. I2C Coverage Analysis

I2C VCS Simulation Result





UVM_WARNING 원인
ACK field가 x로 전달됨
→ Monitor ACK sampling
개선 항목

Scoreboard PASS = 12, FAIL = 0으로 WRITE/READ data path는 정상 확인

단, ACK field가 X로 전달되는 monitor sampling issue로 UVM_WARNING 발생
ACK/NACK coverage는 개선 필요 항목으로 분리

그림 12. I2C VCS Simulation Result

항목	초기 결과	의미
Scoreboard	PASS 12 / FAIL 0	WRITE/READ data path 정상
Total Coverage	85.71%	일부 coverage hole 존재
ACK/NACK Coverage	0%	ACK field가 X로 전달되어 ACK/NACK bin 미도달
UVM_WARNING	12	Monitor ACK sampling 개선 필요

5.4 Waveform 분석

SPI waveform에서는 CS_N Low 구간이 transaction active window로 유지되고, 해당 구간 안에서 8개의 SCLK cycle이 발생하는지 확인하였다. MOSI는 Master TX가 Slave RX로 전달되는 경로이고, MISO는 Slave TX가 Master RX로 전달되는 경로이다. I2C waveform에서는 START, WRITE address, ACK, WRITE data, repeated START, READ address, ACK, READ data, NACK 순서가 protocol sequence와 일치하는지 확인하였다.

SPI Waveform Check Point



Check Point		
①	CS_N Low	Transaction active window (Slave Select)
②	8 SCLK Cycles	8-bit SPI transfer
③	MOSI	Master TX → Slave RX
④	MISO	Slave TX → Master RX

그림 13. SPI Waveform Check Point

I2C Waveform Check Point

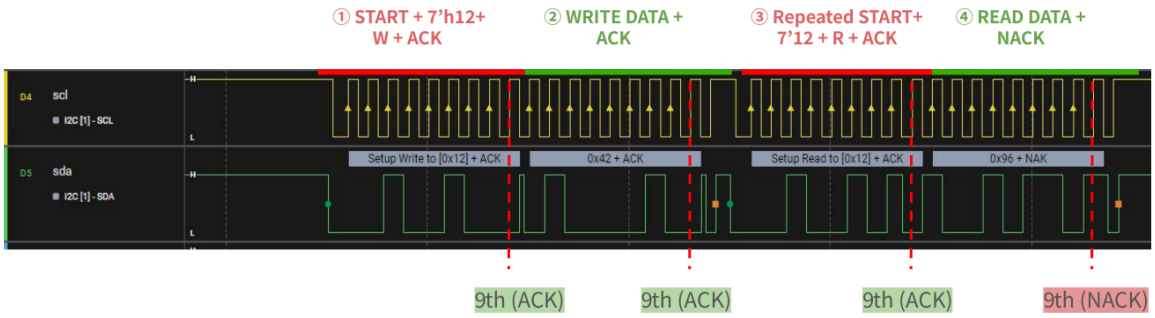


그림 14. I2C Waveform Check Point

6. 결과 분석 및 트러블슈팅 (Analysis & Troubleshooting)

6.1 SPI Coverage 분석

SPI는 Scoreboard 기준으로 모든 transaction이 PASS하였지만, Functional Coverage는 59.59%에 머물렀다. 이는 설계 기능 오류라기보다 directed pattern 중심의 test sequence로 인해 일부 data pattern bin이 도달하지 못한 결과이다. 따라서 SPI 검증은 data path 정상성은 확인되었으나, coverage closure 관점에서는 constrained-random sequence와 master_tx x slave_tx cross coverage 확장이 필요하다.

- 검증 완료: CS_N Low 구간, 8 SCLK cycle, MOSI/MISO full-duplex data match
- Coverage hole: 일부 data pattern bin 미도달
- 개선 방안: constrained-random item 생성, seed 반복 수행, data cross coverage 추가

6.2 I2C ACK Sampling Issue

I2C 검증 중 slave_ack_received 값이 X로 전달되고, cp_ack coverage가 0%로 남는 문제가 발생하였다. Scoreboard의 WRITE/READ data path는 PASS였지만, ACK/NACK coverage가 도달하지 않아 Monitor의 protocol timing 복원 기준을 재검토하였다.

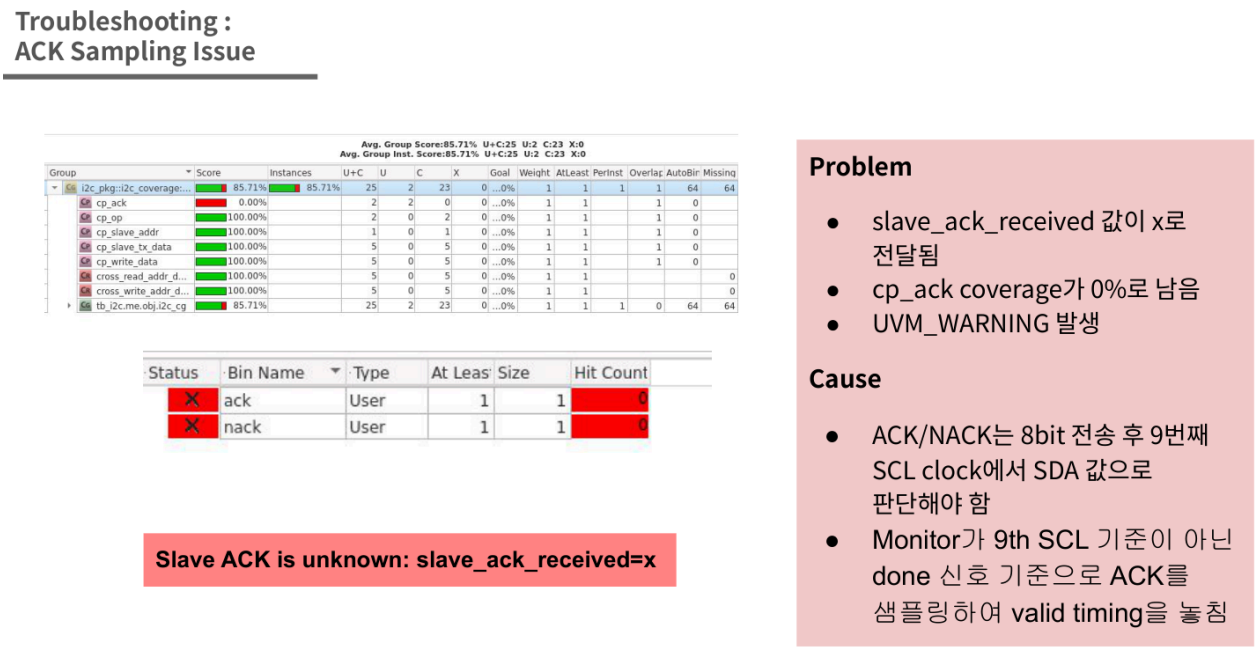


그림 15. Troubleshooting - ACK Sampling Issue

구분	내용
Problem	slave_ack_received 값이 X로 전달됨, cp_ack coverage가 0%로 남음, UVM_WARNING 발생
Root Cause	ACK/NACK는 8-bit 전송 후 9번째 SCL clock에서 SDA 값으로 판단해야 하나, 기존 Monitor는 done 신호 기준으로 ACK를 sampling하여 valid timing을 놓침
Debug Point	9th SCL edge 기준 SDA value, ACK=0/NACK=1 field, transaction item 전달 여부 확인
영향	Data path는 PASS였지만 ACK/NACK coverage가 미도달하여 검증 완성도 저하

6.3 개선 결과

수정 후 Monitor는 9번째 SCL clock 기준으로 SDA를 직접 sampling하고, ACK=0 및 NACK=1 값을 transaction item에 저장하도록 개선하였다. 또한 Scoreboard와 Coverage에 ACK field를 전달하고, wrong address/NACK negative test PASS 조건을 추가하였다. 그 결과 cp_ack 및 cp_nack bin이 hit 되었고, Functional Coverage는 100%, Scoreboard는 PASS 14 / FAIL 0으로 개선되었다.

Troubleshooting : Improvement & Result

Before Code

```
do begin
    @(vif.mon_cb);
end while (vif.mon_cb.master_done !== 1'b1);

@(vif.mon_cb);
item.slave_ack_received = vif.mon_cb.slave_ack_received;
```

Fix Code

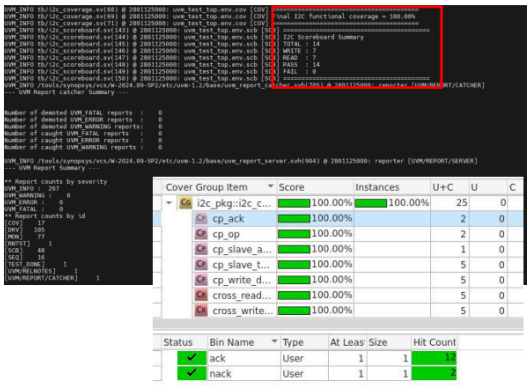
```
task automatic sample_ack_from_sda(output logic ack_value);
    // I2C ACK/NACK는 8bit 전송 이후 9번째 SCL clock에서 결정됨
    // ACK  = SDA Low  = 0
    // NACK = SDA High / Release = 1

    repeat (9) begin
        @(posedge vif.scl);
    end

    #1step;

    if (vif.sda === 1'b0) begin
        ack_value = 1'b0;   // ACK
    end else begin
        ack_value = 1'b1;   // NACK or released bus
    end

    `uvm_info("MON", $sformatf(
              "ACK/NACK sampled from SDA: sda=%b ack_value=%0b",
              vif.sda, ack_value), UVM_MEDIUM)
endtask
```



Improvement

- 9th SCL clock에서 SDA 직접 sampling
- ACK = 0, NACK = 1로 item에 저장
- Scoreboard / Coverage에 ACK field 전달
- Wrong address / NACK negative test PASS 조건 추가

Result

- cp_ack / cp_nack hit
- Functional Coverage = 100%
- PASS = 14 / FAIL = 0
  - 기존 12 + ACK/NACK 및 wrong address negative scenario를 추가
- UVM_ERROR = 0 / UVM_FATAL = 0

그림 16. Troubleshooting - Improvement & Result

항목	개선 전	개선 후
ACK/NACK sampling	done 신호 기준 sampling	9th SCL clock 기준 SDA 직접 sampling
ACK field	X 전달	ACK=0, NACK=1로 item 저장
Coverage	ACK/NACK coverage 0%, Total 85.71%	cp_ack/cp_nack hit, Functional Coverage 100%
Scoreboard	PASS 12 / FAIL 0	PASS 14 / FAIL 0
Severity	UVM_WARNING 발생	UVM_ERROR 0 / UVM_FATAL 0

7. 결론 및 고찰

7.1 결론

본 프로젝트에서는 SPI/I2C RTL을 대상으로 UVM 기반 검증 환경을 구성하고, protocol-aware transaction을 이용한 자동 검증을 수행하였다. SPI는 CS_N Low 구간에서 8-bit Full-Duplex transfer가 정상적으로 이루어지는지 확인하였고, I2C는 START/STOP, address, ACK/NACK, WRITE/READ data path를 중심으로 검증하였다. Scoreboard와 Functional Coverage를 통해 단순 파형 확인을 넘어 transaction 단위의 검증 결과를 정량적으로 확인할 수 있었다.

성과	내용
UVM 환경 구축	SPI/I2C 각각의 Sequence, Driver, Monitor, Scoreboard, Coverage 구성
SPI 검증	Full-Duplex data path PASS, Scoreboard PASS 48 / FAIL 0
I2C 검증	WRITE/READ data path PASS, ACK/NACK sampling issue 개선 후 Coverage 100%
Troubleshooting	ACK sampling 기준을 9th SCL로 수정하여 Monitor의 protocol timing 복원 정확도 개선
FPGA Validation	Basys3 board-to-board 및 Logic Analyzer를 통해 실제 pin waveform 확인

7.2 고찰

이번 프로젝트를 통해 검증에서 중요한 것은 DUT가 한 번 정상 동작하는 파형을 확인하는 것이 아니라, 어떤 기준으로 PASS/FAIL을 판단하고 어떤 scenario가 coverage로 확인되었는지를 명확히 정의하는 것임을 확인하였다. 특히 I2C ACK sampling issue를 해결하는 과정에서 Monitor는 단순히 done 신호를 보는 것이 아니라 protocol timing에 맞춰 bus signal을 정확히 transaction으로 복원해야 한다는 점을 배웠다.

- Scoreboard PASS만으로는 충분하지 않으며, Functional Coverage를 함께 분석해야 검증 공백을 확인할 수 있다.
- Protocol-aware Monitor 설계가 중요하며, I2C처럼 timing-sensitive한 bus에서는 ACK/NACK sampling 시점이 검증 정확도에 직접적인 영향을 준다.
- SPI coverage hole처럼 설계 오류가 아니더라도 test scenario 부족으로 coverage가 낮아질 수 있으므로, constrained-random과 cross coverage 확장이 필요하다.
- Simulation 결과와 FPGA board waveform을 비교하면 실제 하드웨어 환경에서 배선, reset, pull-up, signal observation 등 추가 이슈를 확인할 수 있다.

7.3 추후 개선 방향

개선 항목	개선 방향
SPI Coverage Closure	Constrained-random sequence, master_tx x slave_tx cross coverage, CPOL/CPHA mode 확장 검증
I2C Scenario 확장	Repeated START, multi-byte write/read, clock stretching, arbitration loss 등 확장 scenario 추가
Assertion 기반 검증	SPI SCLK/CS_N timing, I2C START/STOP/ACK timing에 대한 SVA 추가
Regression 자동화	Seed 기반 random regression, coverage merge, pass/fail report 자동화
FPGA 검증 고도화	Logic Analyzer capture 결과와 simulation waveform 비교 기준 정량화